Supplementary Material S1: Formal verification of the switching criterion

Global Convergence and Reliable Stopping in a Safeguarded Modified Anderson–Björck Method

Latchezar Tomov and Nedelcho Ganchovski

This supplement contains the material on the Lean 4 formalization and the supporting numerical checks that is summarized in Section 6 of the main article. Numbers with the prefix S refer to this supplement; all other numbers of equations, lemmas, theorems and sections refer to the main article.

The proof has been checked in Lean 4 using the real-number theory of mathlib and the floating-point definitions and theorems of FloatLib [1, 2]. Its first task is to establish the local error bounds from a concrete definition of rounding. Its second task is to show that the operations on binary64 representations have exactly the real values described by this rounding map. Both steps are needed: an estimate obtained by propagating assumed local errors would leave open whether those assumptions hold for the implemented operations.

S1 The rounding map and the two divisions by two

For $x \neq 0$, let $m(x)$ be the unique integer satisfying

$$2^{m(x)-1} \leq |x| < 2^{m(x)},$$

and put

$$e(x) = \max\{m(x) - 53, -1074\}, \qquad \mathrm{rn}(x) = 2^{e(x)} N(x 2^{-e(x)}), \qquad \mathrm{rn}(0) = 0. \tag{S1}$$

Here $N(t)$ is the nearest integer to t, with the even integer chosen when the two distances are equal. The lower bound on $e(x)$ makes the smallest spacing equal to $\omega = 2^{-1074}$. It therefore includes subnormal numbers and rounding to zero without imposing a relative error estimate near zero.

The map in (S1) has no upper exponent bound. For a value to be a finite binary64 number, it must also satisfy

$$|x| \leq \Omega = (2^{53} - 1)2^{971}. \tag{S2}$$

This additional condition is checked when the real-valued map is related to the finite format. It is not inferred merely from the precision of the significand.

Lemma S1 (Absolute rounding error on the required range)**.** *The map* rn *is nondecreasing and fixes every number representable in its binary grid. Moreover,*

$$|x| \leq 2 \quad \Longrightarrow \quad |\,\mathrm{rn}(x) - x| \leq \epsilon, \qquad |\,\mathrm{rn}(x)| \leq 2. \tag{S3}$$

Consequently, every rounded value in this statement is finite in binary64.

Proof. Monotonicity and preservation of representable values are established for the rounding map in the underlying formal theory. To obtain the error bound needed here, suppose first that

$0 < |x| < 2$. Then $m(x) \le 1$ and hence $e(x) \le -52$. Since N changes its argument by at most $1/2$,

$$|\operatorname{rn}(x) - x| \le \tfrac{1}{2}\, 2^{e(x)} \le 2^{-53} = \epsilon.$$

The remaining values 0, 2 and -2 are representable and are unchanged by rounding. Monotonicity, applied between the fixed points -2 and 2, gives the second inequality in (S3). Finally, $2 < \Omega$, so the finite-range condition is satisfied. Each of these steps is checked in the Lean proof; the subnormal range is covered by the same absolute estimate. □

There are two divisions by two in (49)–(50), and their treatment is different. The division of $\widehat{a}$ by two is exact. The division used to form $\widehat{h}$ may round, since its numerator can be as small as a subnormal number. For example, $\operatorname{rn}(\omega/2) = 0$. The formal evaluation retains a rounding operation at both places and proves that the first one can be removed.

Lemma S2 (Exact halving in the symmetry factor)**.** *If* $0 \le \rho \le 1$, $\widehat{\rho} = \operatorname{rn}(\rho)$ *and* $\widehat{a} = \operatorname{rn}(1 - \widehat{\rho})$, *then*

$$\operatorname{rn}(\widehat{a}/2) = \widehat{a}/2. \tag{S4}$$

Proof. Monotonicity gives $0 \le \widehat{\rho} \le 1$. If $\widehat{\rho} = 1$, then $\widehat{a} = 0$ and the assertion follows. Otherwise, the spacing immediately below 1 is 2^{-53}, so

$$1 - \widehat{\rho} \ge 2^{-53} = \epsilon.$$

More explicitly, every representable $x \in [1/2, 1)$ is an integer multiple of ϵ. The strict inequality $x < 1 = 2^{53}\epsilon$ therefore gives $x \le (2^{53} - 1)\epsilon$. For $x < 1/2$ the same upper bound is immediate. This proves that there is no representable number strictly between $1 - \epsilon$ and 1.

Since ϵ is itself representable, monotonicity now gives $\widehat{a} \ge \operatorname{rn}(\epsilon) = \epsilon$. Also $\widehat{a} \le 1$. Dividing such a representable number by two lowers its exponent by one and preserves its significand; the result is at least 2^{-54} and remains within the normal range. It is therefore representable and fixed by rounding. The formal proof establishes both the spacing assertion and preservation of representability under this exponent change. □

S2 The finite-format statement

Let $\mathcal{B}$ denote the set of finite binary64 representations and let $\nu : \mathcal{B} \to \mathbb{R}$ assign to each representation its exact real value. The two representations of zero have the same value under ν. Arithmetic on representations is defined with IEEE rounding to nearest, ties to even, including gradual underflow.

For finite operands u, w and an arithmetic operation $\circ$, the formal library supplies the relation between the executable operation and its exact real result. In the range used here, this gives

$$\nu(\operatorname{op}_\circ(u, w)) = \operatorname{rn}(\nu(u) \circ \nu(w)) \quad \text{if} \quad |\nu(u) \circ \nu(w)| \le 2, \tag{S5}$$

where division additionally requires $\nu(w) \neq 0$. The result is proved finite as part of the assertion, so ν is defined on its left-hand side. This is used separately for addition, subtraction, multiplication and division. Absolute value and the selection of a minimum or maximum preserve the corresponding real values exactly.

Write $\mathcal{E}_{64}(w_1, w_2, w_3)$ for the binary64 representation obtained by evaluating the scaled difference in the stated order, including the final subtraction. Write $\mathcal{E}_{\mathrm{rn}}(f_1, f_2, f_3)$ for the real-valued expression obtained by placing rn at the same arithmetic operations. In particular, it initially contains $\operatorname{rn}(\widehat{a}/2)$; its simplification uses Lemma S2.

Theorem S1 (Verified correspondence and switching decision)**.** *Let $w_1, w_2, w_3 \in \mathcal{B}$, put $f_i = \nu(w_i)$, and suppose $f_1 f_2 < 0$. Define ρ, s, p_i, h, G, L, R as in (44)–(48). Every intermediate arithmetic result in the evaluation of $\mathcal{E}_{64}$ is finite, and every divisor is positive. Furthermore,*

$$\widehat{G} := \nu(\mathcal{E}_{64}(w_1, w_2, w_3)) = \mathcal{E}_{\mathrm{rn}}(f_1, f_2, f_3), \qquad |\widehat{G} - G| \le 25\epsilon. \tag{S6}$$

The executable comparisons with the binary64 representations of 2^{-48} and -2^{-48} consequently satisfy

$$\widehat{G} > 2^{-48} \implies L < R, \qquad \widehat{G} < -2^{-48} \implies R < L.$$

They select the respective branches whenever $G > 57\epsilon$ or $G < -57\epsilon$.

Proof. The hypothesis implies that the endpoint magnitudes and their maximum are positive. It follows that $s > 0$, $0 < \rho \le 1$ and $|p_i| \le 1$. The normalized endpoint values have opposite signs. Their rounded values retain the weak sign inequalities even if one of them becomes zero, and hence $|\widehat{p}_1 + \widehat{p}_2| \le 1$.

Monotonicity of rn gives the intervals for the computed factor stated in the proof of Theorem 3, as well as $|\widehat{h}| \le 1/2$. The exact results to which rounding is applied all belong to $[-2, 2]$. The endpoint maximum and s are positive, $\widehat{b} \ge 1$, and the remaining divisor is 2. These facts are established before using (S5).

Apply that relation successively to the arithmetic subexpressions. For the initial input expressions, equality of real values is the definition of f_i. If equality has already been established for the operands of an operation, its range bound and, where needed, its positive divisor allow (S5) to be applied. The new representation is finite and its value agrees with the corresponding rounded real expression. Induction over the expression therefore proves the equality in (S6) and finiteness of all intermediate results.

Lemma S1 now supplies each local absolute error bound, and Lemma S2 supplies the exact first halving. The formal real-arithmetic argument then derives the bounds with constants 8, 3, 5, 19 and 25 used in Theorem 3. These bounds are conclusions of the proof, not additional assumptions on the calculation. For example, the change in κ is bounded algebraically by

$$\kappa(x) - \kappa(y) = (x - y)\frac{1 + 2x + 2y + 3xy}{(1 + x)^2(1 + y)^2}, \qquad x, y \ge 0.$$

The fraction lies in $[0, 1]$, which proves $|\kappa(x) - \kappa(y)| \le |x - y|$ and supplies the estimate used in the propagation of errors.

Finally, $2^{-48} = 32\epsilon$ is exactly representable. The comparisons on finite binary64 representations are proved to agree with the strict order of their real values. Applying the error bound gives the implications and the 57ϵ separation conditions, as in Corollary 3. The identity $G = (R - L)/s$, with $s > 0$, returns the assertion to the original unscaled inequality. □

S3 Correspondence with the C# expression and reproducibility

The accompanying C# implementation of the scaled comparison uses separate addition, subtraction, multiplication and division calls. A small source translator records their arithmetic expressions in Lean, preserving the order and nesting of all 17 operations. Equality with the rounded expression used in the proof is then checked in Lean. In particular, the two products in the right-hand side are rounded separately before their sum is formed. The decimal constant used for the comparison is also checked to equal 2^{-48} exactly. The real-value correspondence is sufficient for the switching criterion since the distinction between positive and negative zero cannot change the comparison, and none of the divisors can be zero.

Version 0.5.0 of the development uses Lean 4.34.0 and fixed revisions of mathlib and FloatLib, recorded in the project. Its audit covers 380 lemmas and theorems and checks their transitive axiom dependencies. They use only propositional extensionality, the quotient principle and

classical choice from the standard logical foundation [1]. There are no unfinished proofs or additional axioms asserting the correctness of rounding. The source files, fixed dependency revisions, source checksums and verification records are retained with `ModAB.Switching.Lean`, version 0.5.0 (fixed source revision).

The relation to native execution has a definite scope. The source translator is an inspected component, not a formally verified C# compiler. Lean checks the arithmetic expression that it produces and the executable IEEE model of that expression. Application of the result to a native C# execution additionally requires the compiler, .NET runtime and processor to perform the stated separate binary64 operations, with the rounding and gradual-underflow assumptions of Section 5.5. Their complete implementations are outside the formal proof.

A separate C# verification checks these arithmetic assumptions on the recorded execution environment, .NET 10.0.12 on Linux x64. For the 23,816 input triples described above, all 404,872 arithmetic operations were compared with independent rounding based on exact integer quotients, remainders and parity. A further 14,268 operations exercise boundary cases, including signed zeros, subnormal numbers, normal-range boundaries, ties and overflow. All 419,140 comparisons agree in all 64 bits. These checks support the correspondence for the tested environment; the universal error bound follows from the formal proof.

The verified assertion concerns admissible inputs to the scaled comparison. The C# rejection of invalid inputs is covered by separate tests. Accuracy of the supplied values of the function, the entire floating-point root-finding implementation and the native compiler are not thereby verified. The module `Unscaled.lean` proves Proposition 4 by propagating errors through the specified sequence of concrete rounding operations under its stated normal-range and no-overflow assumptions. This rounded-real proof does not establish the native C# execution or a separate finite-format correspondence for every operation in the unscaled expression. The scaled comparison remains the separate arithmetic variant specified above, and its verification does not alter the benchmark results for the other implementations.

S4 Reference evaluation in exact rational arithmetic

A separate Python reference evaluation checked 23,816 input triples against exact rational arithmetic, including extreme exponents, subnormal values, points inside the switching region, and neighboring binary64 numbers at its exact boundary. All five bounds used in the proof, with constants 8, 3, 5, 19 and 25, held in these checks, and every decision certified by the guard agreed with the exact comparison. The same calculation verified (46) on the 22,615 triples satisfying its normal-range assumptions. These are finite checks of the formulas and their reference implementation; the universal claims follow from the proofs above. The concentration of cases near the boundary is deliberate and gives no estimate of how often rounding changes a decision in a practical root-finding problem.

References

[1] Avigad, J.; de Moura, L.; Kong, S.; Ullrich, S. *Theorem Proving in Lean 4*. Online edition, with contributions from the Lean Community. `https://lean-lang.org/theorem_proving_in_lean4/` (accessed on 5 October 2026).

[2] FloatLib contributors. *FloatLib: Verified Floating-Point Arithmetic in Lean*. Revision `1e83f09ed8c41a953cf8f93d26c210778177b94a`, 2026. Available online: fixed source revision (accessed on 5 October 2026).